CIÊNCIAS EXATAS E DA TERRA

ANÁLISE E DESENVOLVIMENTO DE SISTEMAS - TECNÓLOGO

LUCAS ANTONIO DA SILVA

PORTIFÓLIO INDIVIDUAL:

Desenvolvimento em Javascript

São Paulo
2025

LUCAS ANTONIO DA SILVA

PORTIFÓLIO INDIVIDUAL:
Desenvolvimento em Javascript

Trabalho de portfólio apresentado como requisito parcial para a obtenção de média semestral.

Orientadores: Alexandro e Tiago Barroso dos Santos

São Paulo
2025

SUMÁRIO

1. INTRODUÇÃO 3
2. DESENVOLVIMENTO 4
3. CONCLUSÃO 9
4. REFERÊNCIAS 10

1. INTRODUÇÃO

A validação de formulários é um componente crítico no desenvolvimento web moderno. Ela assegura que os dados inseridos pelo usuário sejam corretos e estejam no formato esperado antes de serem processados por um sistema. A validação pode ocorrer no lado do servidor ou no lado do cliente.

A validação no lado do cliente, realizada diretamente no navegador do usuário, oferece a vantagem de um feedback imediato, melhorando significativamente a experiência do usuário (UX) ao evitar a necessidade de recarregar a página para informar sobre um erro. A linguagem JavaScript é a ferramenta padrão para executar essa validação.

O objetivo deste relatório é documentar o processo de criação de um formulário HTML simples e a implementação de um script JavaScript para a validação em tempo real de um campo de e-mail, conforme a atividade prática proposta. O ambiente de desenvolvimento utilizado foi o editor online Playcode.io.

.

2. DESENVOLVIMENTO

Para a execução da atividade, foi utilizado o editor de código online Playcode.io, que permite a criação de projetos web (HTML, CSS e JavaScript) sem a necessidade de configuração de ambiente local.

O processo foi dividido nas seguintes etapas:

Estruturação: Foi criado o arquivo index.html. Nele, foi implementado um formulário (<form>) contendo um campo de entrada (<input type="text">) para o e-mail e um botão de envio (<button type="submit">). Uma tag <p> vazia foi adicionada para servir como contêiner da mensagem de feedback.

index.html × script.js × style.css × Web View ×

```html
<!DOCTYPE html>
<html lang="pt-br">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Validação de E-mail</title>
    <link rel="stylesheet" href="src/style.css" />
  </head>
  <body>
    <div class="container">
      <h2>Formulário de Validação</h2>
      <p>Insira um e-mail para validar o formato.</p>

      <form id="validarFormulario" novalidate>
        <label for="email">E-mail:</label>
        <input type="text" id="emailInput" placeholder="exemplo@dominio.com" />

        <button type="submit">Validar</button>
      </form>

      <p id="mensagemErro" class="mensagem"></p>
    </div>
                Follow link (ctrl + click)

    <script src="src/script.js"></script>
  </body>
```

.

Estilização: Um arquivo style.css foi criado para prover uma interface visual limpa e destacar as mensagens de erro (na cor vermelha) e sucesso (na cor verde), que por padrão ficam ocultas.

index.html × script.js × style.css × Web View ×

```
1   body {
2     background: transparent;
3     color: #fcbe24;
4     padding: 0 24px;
5     margin: 0;
6     height: 100vh;
7     display: flex;
8     justify-content: center;
9     align-items: center;
10    font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, Oxygen,
11  }
```

Implementação da Lógica: No arquivo script.js, a lógica de validação foi implementada:

- **Seleção de Elementos:** O script primeiro seleciona os elementos HTML (formulário, campo de input e parágrafo da mensagem) através de seus IDs.
- **Prevenção de Envio Padrão:** Foi adicionado um "ouvinte de evento" (addEventListener) ao formulário para interceptar o evento submit. A função event.preventDefault() foi utilizada para impedir que o formulário recarregasse a página, permitindo que o script assumisse o controle.
- **Lógica de Validação:** Utilizou-se uma Expressão Regular (RegEx) – /^[^\s@]+@[^\s@]+\.[^\s@]+$/ – que define o padrão estrutural de um e-mail (caracteres, seguido de @, seguido de mais caracteres, um . e caracteres finais).
- **Feedback ao Usuário:** O valor do input é testado contra a RegEx. Se o teste for positivo, uma mensagem de sucesso é exibida. Se for negativo, o script exibe a mensagem de orientação solicitada, informando a estrutura correta de um e-mail e aplicando a classe CSS de erro.

index.html × script.js × style.css × Web View ×

```javascript
document.addEventListener('DOMContentLoaded', () => {


    const formulario = document.getElementById('validarFormulario');
    const inputEmail = document.getElementById('emailInput');
    const mensagemErro = document.getElementById('mensagemErro');

    formulario.addEventListener('submit', (evento) => {

        evento.preventDefault();

        const email = inputEmail.value.trim();

        const regexEmail = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

        if (regexEmail.test(email)) {
            mensagemErro.textContent = 'Formato de e-mail válido!';
            mensagemErro.className = 'mensagem sucesso'; // Classe CSS para sucesso
        } else {
            mensagemErro.textContent = 'E-mail inválido. Um e-mail deve seguir o formato "nome@provedor.com"
            mensagemErro.className = 'mensagem erro'; // Classe CSS para erro
        }
    });
});
```

2.1. RESULTADOS E TESTES

O projeto foi implementado com sucesso no Playcode.io. A interação do usuário com o formulário produziu os dois cenários de resultados esperados:

Cenário 1: E-mail Inválido Ao inserir dados que não correspondem ao formato de e-mail (ex: "teste", "teste@" ou "teste.com") e clicar no botão "Validar", o script executa as seguintes ações:

- A página **não** é recarregada.
- A Expressão Regular retorna false (falso).
- A mensagem de orientação é exibida em vermelho: "E-mail inválido. Um e-mail deve seguir o formato "nome@provedor.com" (ex: conter um "@" e um ".")."

Cenário 2: E-mail Válido Ao inserir dados que correspondem ao formato de e-mail (ex: "usuario@exemplo.com") e clicar no botão "Validar":

- A página **não** é recarregada.
- A Expressão Regular retorna true (verdadeiro).
- Uma mensagem de sucesso é exibida em verde: "Formato de e-mail válido!"

O uso do input type="text" (em vez de type="email") foi intencional para garantir que a validação fosse executada exclusivamente pelo script JavaScript, conforme o objetivo do exercício, e não pelo navegador. A abordagem demonstrou ser eficaz em fornecer feedback instantâneo e claro ao usuário.

3. CONCLUSÃO

O desenvolvimento desta atividade prática permitiu aplicar com sucesso os conceitos fundamentais do JavaScript na manipulação do DOM (Document Object Model) e na validação de formulários no lado do cliente.

Conclui-se que a utilização de JavaScript para validação de entradas é essencial para a criação de interfaces web interativas e amigáveis. A capacidade de fornecer feedback imediato ao usuário, orientando-o sobre o preenchimento correto dos campos, reduz a frustração e melhora a integridade dos dados coletados pelo sistema. O objetivo da aula prática foi plenamente atingido.

4. REFERÊNCIAS

HAVERBEKE, Marijn. **Eloquent JavaScript**. 3. ed. [S.l.]: No Starch Press, 2018. Disponível em: <https://eloquentjavascript.net/.> Acesso em: 29 out. 2025.

FLANAGAN, David. **JavaScript**: o guia definitivo. 7. ed. São Paulo: Editora Bookman, 2020.

JQUERY. **JQuery API Documentation**. The JQuery Foundation, 2025. Disponível em: <https://api.jquery.com/>. Acesso em: 28 out. 2025.